

MRV & Ops Copilot – MVP-01 Project Overview

Problem

Modern data-driven organizations, especially in blockchain and DevOps contexts, struggle to **integrate and trust diverse event streams**. On-chain events (e.g. blockchain transactions, identity token minting) and off-chain events (script outputs, GitHub actions, manual inputs) often remain siloed. This fragmentation makes it hard to **measure contributions, verify operations, and report outcomes** in a unified way. Moreover, without a consistent process, **operational metrics and trust scores** can be inaccurate or easily gamed (as cautioned by Goodhart's Law: *"When a measure becomes a target, it ceases to be a good measure"* ¹). The result is a lack of **transparency and efficiency**: teams cannot clearly see who contributed what, and management lacks reliable indicators (e.g. pipeline health or member "karma") to inform decisions.

Solution

MVP-01: MRV & Ops Copilot is a Python-based service designed to **collect, validate, and analyze events across systems** to produce meaningful metrics and assist operations. "MRV" stands for **Measurement, Reporting, and Verification** – the system measures activity, reports daily summaries, and verifies contributions via witness attestations. "Ops Copilot" denotes an assistant for operational oversight. The MVP solution will:

- **Ingest events** from multiple sources (both on-chain and off-chain) in real-time or batch, using a standard newline-delimited JSON format (NDJSON/JSONL) for each event ². Each event carries a timestamp, source, type, key identifier, payload, and an optional list of witnesses (people or entities confirming the event).
- **Validate and store events** in a **local analytic database** (DuckDB) for querying. Events conform to a canonical **JSON Schema (draft-07)**, ensuring structure and data types are consistent ³. We leverage Python's `jjsonschema` library for validation and Pydantic models for parsing and schema generation ⁴. This guarantees that only well-formed events enter the system, and the schema (being draft-07) is widely supported by tools for future integration ³.
- **Compute a "Karma" score** (v0) for identities based on their events. The score algorithm combines **base points for each event**, plus **weights for witness confirmations and event frequency**, and applies a **decay factor for recency**. The decay is implemented as an **Exponentially Weighted Moving Average (EWMA)** so that recent events boost karma more than old events (older contributions "fade" over time). This approach ensures active contributors maintain high scores while inactive ones' scores gracefully diminish, providing a time-sensitive reputation metric ⁵. All parameters (weights, half-life for decay, etc.) are configurable. This **Karma system** quantifies contributions in a way that is harder to game – incorporating qualitative "witness" input as a sanity check to mitigate Goodhart's Law effects (i.e., including human feedback so that purely gaming numeric metrics is less effective) ¹.
- **Generate daily reports** in Markdown summarizing the day's activity and insights. Each daily report will include event counts, number of new or completed missions, top N identities by Karma score, and a brief changelog or "operational proposals" section. This provides stakeholders a concise overview of progress and any issues. The report is not only saved locally

but can also be **published via IPFS** (optional), by adding the report to IPFS and pinning it, yielding a content-addressed CID for distributed access (ensuring tamper-proof archival).

- **Maintain an audit trail** in an append-only `witness.jsonl` log (“memoria”). Every daily report generation appends a special event entry (kind: `daily_report`) with a summary of stats. This serves as a verifiable memory of what the system calculated each day, aiding transparency and debugging.
- **Adhere to DevOps/SRE best practices** for pipeline health: The Copilot tracks key **SLIs/SLOs** (Service Level Indicators/Objectives) such as event ingestion latency and validation success rate. For example, we might set an SLO that 99% of daily reports are generated before midnight UTC, allowing a 1% *error budget* for delays ⁶. We also plan to connect the system with **DORA “Four Keys” metrics** (Deployment Frequency, Lead Time for changes, Change Failure Rate, Mean Time to Restore) ⁷ ⁸ once integrated with CI/CD, to continuously improve the development process itself. Essentially, the MVP not only produces metrics for others, but also keeps metrics on its own performance (pipeline observability).

By combining these capabilities, the **MRV & Ops Copilot** solution addresses the problem: it creates a **single source of truth for events and contributions**, calculates a credible **reputation score (Karma)** for identities, and **provides daily operational insight**. All of this is built on open standards and is easily extensible, laying the groundwork for an AI-driven “copilot” that can eventually make recommendations (e.g. flag anomalies or suggest next actions in operations).

Project Plan and Implementation

To deliver this MVP, we have defined clear objectives, scope, deliverables, and acceptance criteria. Below is a breakdown of the project’s key components and the implementation strategy:

Objective

Build a self-contained Python service (package `mrvcopilot`) that **ingests events, computes Karma scores, and outputs daily reports**, enabling an initial “copilot” for monitoring and guiding operations. This includes establishing the data schema, storage, computation logic, and output formats as described in the Solution. The service should run locally (for now) and be easy to configure or extend.

Scope & Key Decisions

Several **technical decisions** have been made to ensure the MVP is robust and aligned with industry best practices:

- **Event format – JSON Lines:** Incoming events will be provided in **newline-delimited JSON (JSONL/NDJSON)** files, where each line is a separate JSON object ². This format is convenient for streaming and incremental processing (one event per line, no complex parsing of large JSON arrays). It also allows the system to handle partial failures gracefully (e.g., one bad line doesn’t break the whole file) ⁹. We chose NDJSON for its simplicity and compatibility with many tools.
- **Canonical schema – JSON Schema Draft-07:** We define a schema for events using JSON Schema draft-07, which is widely adopted and supported by validators ³. Draft-07 was chosen because many libraries (including Python’s `jsonschema` and Pydantic) have mature support for it, making validation and future schema evolution easier. The schema enforces required fields (`event_id`, `ts`, `source`, `kind`, `key`, `payload`) and correct data types (e.g., `ts` as date-time string, `source` as an enumerated string, etc.). This ensures **data quality** at ingestion. (Note: We will use the `format` keyword for the timestamp in ISO 8601, but since

strict format validation can vary, we may supplement it with a regex pattern if needed for precision ¹⁰ ¹¹).

- **Validation and modeling – jsonschema + Pydantic:** Every event line is validated against the JSON Schema using the `jsonschema` library in Python (which supports Draft-07). Additionally, we leverage **Pydantic v2** to create Python data models corresponding to the schema, which provides automatic parsing and validation. Pydantic can even auto-generate the JSON Schema from the models, ensuring our documentation stays in sync ⁴. This dual approach (`jsonschema` for raw validation, `Pydantic` for convenient data handling) gives both **rigor and developer productivity**.
- **Storage – DuckDB local warehouse:** For storing and querying events, we use **DuckDB**, an in-process analytical database. DuckDB was selected for its excellent performance on analytical queries on a single machine and zero-setup usage (just a file on disk) ¹². The events and derived tables (karma scores, daily summaries) will reside in a local DuckDB database file (e.g. `artifacts/mrv.duckdb`). DuckDB can efficiently handle **large datasets (up to billions of rows)** on one machine ¹², reading directly from disk when data doesn't fit in RAM, which means this solution will scale well in terms of data volume before we'd need to consider a distributed database. We avoid heavy external dependencies – DuckDB requires no dedicated server and minimal configuration, fitting the MVP needs.
- **Efficient ingestion – bulk loading:** To ingest JSON lines into DuckDB, we avoid inserting one row at a time (which would be slow for large files). Instead, we prefer bulk loading using DuckDB's native JSON functions. For example, DuckDB supports reading a JSON file directly via `read_json_auto('file.json')` or using the `COPY` command with `FORMAT 'json'` ¹³. This approach is **much more efficient** than looping over inserts, especially for big event dumps. The ingest process will likely accumulate incoming events and then perform a batch load. We also implement an **idempotent consumer pattern**: by keeping track of processed `event_id`s (e.g., in a `processed` table with a unique constraint), the ingestor can skip or ignore duplicate events if the same file is processed twice ¹¹. This ensures **no double-counting** – re-running the ingestion on the same data won't corrupt our state, which is critical for reliable operations.
- **Event sourcing model:** The system treats the **event log as the source of truth**, following an event sourcing design ¹⁴ ¹⁵. Every state or outcome (like a Karma score or daily summary) is derived from the append-only log of events, rather than being updated in place arbitrarily. This means we retain a full history, can reconstruct state from scratch if needed, and naturally have an audit trail. This design aligns with Martin Fowler's Event Sourcing principle (storing all changes as a sequence of events) ¹⁴, giving us benefits like the ability to recompute scores with new logic or do temporal queries (e.g., Karma as of last week).
- **Operational observability:** We have built into the plan the collection of **pipeline performance metrics**. Using the event log, we can record how long each stage takes and count errors (e.g., number of validation failures). We define Service Level Indicators such as *ingestion latency*, *report generation success rate*, etc., and set targets (SLOs) to ensure the system meets reliability goals. For instance, "daily report generation success $\geq$ 99% over a month" might be an SLO, implying an error budget of 1% for failures ⁶. We'll document these in the runbook and ensure the system logs timings and errors needed to calculate them. Additionally, we connect with **DORA metrics** (though full CI/CD integration might come later, we are design-ready to incorporate deployment and incident metrics). This focus on measurement for the pipeline itself will help the team continuously improve the MVP's reliability and speed.
- **Dashboarding – Metabase:** To make the data easily visible, we will use **Metabase** (an open-source BI tool) connected to the DuckDB file as a data source. We plan to create at least three starter dashboards ("events per day", "top Karma scores", "missions completed count") so that non-developers can visualize the results. DuckDB's single-file database format is conveniently queryable by Metabase (via a direct connection or through an intermediary if needed), and since

DuckDB is SQL-standard, writing queries for charts is straightforward. This gives a UI aspect to the MVP for demo purposes without having to build a custom frontend.

- **Optional IPFS integration:** Although not strictly required, we include an option to **publish the daily report via IPFS** (using a tool like Kubo, the go-IPFS node). This involves adding the Markdown report file to IPFS, getting its content ID (CID), and possibly pinning it on an IPFS pinning service or our local node ¹⁶. The benefit is an immutable record of each report that can be shared publicly without relying on a centralized server. It's a forward-looking feature to align with decentralized principles (useful if the project's context is a decentralized community/DAO). For the MVP, this will be a simple CLI call or script integration documented for users to run manually.

Deliverables (MVP)

The MVP will produce a number of **artifacts** and components for handover:

- **Python Package:** Source code organized as `mrvcopilot/` containing:
 - `schemas/` – JSON Schema definitions (at least `event.schema.json` for the event format) and code for validation (using `jsonschema` library and Pydantic models).
 - `ingest/` – Modules to handle file watching or batch import from `bus/` directory (where `.jsonl` event files arrive). This includes logic to open files, validate each JSON line, deduplicate events, and bulk load into DuckDB.
 - `karma/` – The Karma calculation module implementing **Karma v0** scoring. It reads events (or uses SQL in DuckDB) to aggregate per identity scores, applies weights and decay, and stores the results in a `karma` table with rationale. The configuration for weights and decay constants is externalized (e.g. in `artifacts/mrv_config.yaml`) for easy tuning.
 - `reports/` – Code to generate daily reports. This will likely query the DuckDB (events and karma tables) to produce summary statistics, format them into a Markdown template, and save to `reports/daily/YYYY-MM-DD.md`. After generating, it also appends a summary line to the `memoria/witness.jsonl` log.
- (Additionally, a `__main__.py` or CLI might be provided to run the whole pipeline easily, and possibly a scheduler or simple loop to trigger daily runs.)
- **DuckDB database:** A file `artifacts/mrv.duckdb` that contains the following tables:
 - `events`: the primary event log table with columns (timestamp, source, kind, key, event_id, and the full JSON blob or parsed fields). This table will grow as events are ingested. A uniqueness constraint on `event_id` (or a separate `processed` tracking table with primary key) ensures no duplicates.
 - `karma`: a table of the latest Karma scores per identity (id, score, last_updated, rationale). This table is regenerated or updated whenever new events come in that affect scores.
 - `daily`: a daily summary table with one row per date, capturing counts of certain event types (e.g., number of “claims” or missions completed) and maybe high-level metrics of that day.
 - `processed`: (if used for idempotency) a table listing processed event_ids and perhaps their ingestion timestamp.
- **Configuration files:** e.g. `artifacts/mrv_config.yaml` containing tweakable parameters (weights for witnesses, frequency bonus, decay lambda for EWMA, etc.), and possibly connection info for external systems (if any). This makes the system flexible and **tunable without code changes**.
- **Reports:** Daily Markdown files under `reports/daily/` for each day the pipeline runs. For example, `reports/daily/2025-10-07.md` would be generated with content like:
(Example)

```
# Daily 2025-10-07
```

- events ingested: 2
- unique IDs with new karma: 1
- top scorer: eid:alice:1 (karma 1.9)
- notes: identity eid:alice:1 was created and completed a mission today.

These reports can be used to brief the team or archive progress. They are also appended to the witness log for an audit trail.

- **Memoria (Witness Log):** A newline-delimited JSON log at `memoria/witness.jsonl` which accumulates special events like the generation of reports or administrative actions. Each daily run appends an entry, for example:

```
{"kind": "daily_report", "when": "2025-10-07T23:59:00Z", "events":  
2, "karma_updates": 1}
```

This provides an **immutable record of operational events** performed by the Copilot itself.

- **Metabase Dashboards:** A set of 3 configured questions (cards) in Metabase, which visualize the data from `mrsv.duckdb`. Specifically:
- **Events per Day** – a simple time-series chart showing count of events ingested each day (useful to monitor activity volume).
- **Top Karma Identities** – perhaps a table or bar chart listing the highest current Karma scores and the corresponding IDs (to recognize key contributors or agents).
- **Missions Completed** – assuming “mission.done” is one type of event, a count of missions completed (possibly cumulative or per day) to track how operational goals are being met. We will provide instructions in the README (runbook) on how to connect Metabase to the DuckDB file (DuckDB can be queried via JDBC or through an intermediary connector, and Metabase has built-in support for DuckDB as of recent versions).
- **Documentation (Runbook):** A comprehensive `README_MRV.md` (or documentation site) that serves as a **runbook** for the system. This will include:
 - Setup instructions (e.g. required Python version, how to install dependencies, how to run the pipeline).
 - Configuration guidance (explaining the meaning of each parameter in `mrsv_config.yaml`, how to add a new event source or new schema fields if needed).
 - Troubleshooting tips (common issues like schema validation errors, how to interpret error messages, how to recover from failures).
 - Maintenance tasks (e.g., “how to vacuum or optimize the DuckDB if it grows large”, or rotate out old events if needed).
- **SRE & DORA** details: definitions of our SLOs for this pipeline (e.g., “daily job success rate”), what the error budget policy is (e.g., if more than 1% of daily reports fail in a month, halt new feature deployment until issues are fixed – a typical SRE approach), and mention of DORA metrics we aim to gather as the project evolves (like deployment frequency once we have CI/CD, change failure rate if we measure pipeline breaks, etc.) ¹⁷ ⁸ .
- Security and audit notes: since we are handling potentially sensitive events, note any security considerations (access to the data files, etc.), and how the audit log (witness.jsonl) is to be used in reviews.
- Appendices: any **bibliography** or references for concepts used (JSONL spec, JSON Schema standard, event sourcing, idempotent consumer, EWMA formula, DORA metrics, SLI/SLO definitions, IPFS usage, Tendermint consensus, etc.) to help the developer or future maintainers

understand the background. Many of these are referenced in this document with links for further reading.

Acceptance Criteria

For the MVP to be considered successful, it must meet the following **criteria** in a test environment:

- **Complete Ingestion to Report Pipeline:** Starting from an empty state, the pipeline should be able to **ingest a set of sample events and produce the expected outputs in under 10 minutes**. For example, given two sample events (as in our seed file), running the pipeline results in:
 - a populated DuckDB `events` table with those events,
 - a computed Karma entry for the relevant identity,
 - a Markdown daily report for today's date,
 - and an appended line in the witness log.This demonstrates the end-to-end flow (input → process → output) works.
- **No Duplicate Processing:** The ingestion process must be **idempotent**. If we run the ingestion on the same input file twice, the `events` table should not grow in number the second time. We can verify this by checking that the count of distinct `event_id` in `events` equals the total row count ¹¹ (no duplicates), and/or that the `processed` tracker table prevents re-insertions. This ensures reliability in cases of re-processing or crash recovery.
- **Karma Score Calculation:** The Karma computation should produce **non-zero scores reflecting recent events**. In the sample, after ingesting an identity creation and a mission completion, the identity `eid:alice:1` should have a positive Karma (e.g., base 1.0 + witness bonus + frequency bonus, minus minimal decay for being very recent). If an identity has at least one witness-confirmed event, its Karma should be > 1.0 in our model. We will specifically check that the formula applies recency decay: e.g., an event from today contributes more than an identical event dated a week ago (we can manipulate the timestamp in tests to see the effect). The **rationale** field in the Karma table (e.g., "v0:witness+freq+ewma") should be present, indicating which formula version was used, for traceability.
- **Daily Report Generation:** After running the pipeline, the current date's Markdown report file should be present with plausible content (counts match the ingested events, etc.). The `memoria/witness.jsonl` should contain a new JSON line for the daily report event. In essence, each run of the pipeline on a given day should either update that day's report (if run multiple times on same day, we might append or regenerate, to be decided) and should log exactly one `daily_report` entry per day in the witness log.
- **Metabase Dashboard Connectivity:** It should be possible to connect Metabase to the `mr.v.duckdb` database and view the three prepared sample charts. We consider this criterion met if, during QA, we open Metabase, connect to DuckDB, and the example questions (for events per day, top Karma, etc.) display data without errors. This confirms that the data model is compatible and accessible for external analytics tools (DuckDB supports this out-of-the-box).
- **Performance and Reliability Targets:** While formal load testing is minimal at MVP stage, the system should handle at least a modest burst of events (say, 100 events in a file) quickly and without errors. Validation errors (malformed events) should be caught and not crash the pipeline – those events can be skipped with a warning. The pipeline run for a day (assuming tens of events) should complete in well under the SLO of 60 seconds processing time, and ideally just a few seconds for the small batch (this is expected with local DuckDB operations). We will also verify that the **error budget policy** holds: e.g., if we simulate an error (like an invalid event causing a schema validation failure), the system logs it and continues, and these failures can be counted to ensure they stay below the threshold (e.g., <1% of events rejected, if that's our objective).

Meeting all the above ensures the MVP is not only functional but also **robust to basic real-world conditions** (duplicate data, evolving data, external usage via Metabase, etc.).

Implementation Steps

To achieve the deliverables and meet acceptance criteria, development will proceed through the following **step-by-step tasks**:

1. **Define Schema & Models (Step 5.1):** Create the JSON Schema for events (`schemas/event.schema.json`) following Draft-07 syntax. Include all required fields:
2. `event_id` : string, a unique identifier for the event (used for idempotency).
3. `ts` : timestamp string in ISO 8601 format (we'll use `"format": "date-time"` as a hint, understanding some validators may not enforce it strictly).
4. `source` : string enum of allowed sources (e.g., `"chain"`, `"script"`, `"github"`, `"manual"` – this set can expand).
5. `kind` : string identifying the type of event (e.g., `"eid.minted"` or `"mission.done"`).
6. `key` : string that is the key subject of the event (for example, an ID or resource the event is about, such as an identity ID or mission ID).
7. `payload` : object with additional data (schema can allow arbitrary content here or enforce certain structures based on kind, depending on how detailed we want to be in MVP).
8. `witness` : optional array of strings (identifiers of witnesses who vouch for this event). We will ensure the schema includes appropriate constraints (e.g., if needed, a regex pattern for `event_id` format or specific required fields inside `payload` per kind can be added later). Initially, we'll keep it general to not over-complicate. After writing the schema, implement validation in code: use `jsonschema.validate(instance, schema)` on each JSON line read. In parallel, create a Pydantic BaseModel class representing the event (with the same fields). Pydantic will be configured to use v2 features, and we'll test generating a JSON schema from it to ensure it matches our hand-written one (this also double-validates our design). *Output:* `event.schema.json` file and a Python module (e.g., `schemas/validator.py`) that can be imported to validate events, plus a Pydantic model for events. The schema will be documented so developers know the event format clearly ¹⁸ .

Note: We'll pay attention to date-time validation. The JSON Schema "format" keyword for dates is often not strictly enforced by libraries ¹⁰ , so if we need strict checking, we might use a `pattern` (regex) to ensure `ts` matches the ISO8601 format (e.g., `^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}Z$` for UTC timestamps). This ensures consistency across events.

9. **Implement Ingestion & Storage (Step 5.2):** Develop the component that monitors the `bus/` directory for `.jsonl` files (this could be a simple loop or use Python's `watchdog` to detect new files). When a new event file is found, or on a scheduled run:
10. Open the file, read line by line. For each line, parse JSON and validate against the schema. Invalid lines should be logged and skipped (with an error count).
11. For valid events, check against the `processed` records (e.g., query DuckDB if this `event_id` exists in the `processed` table). If already seen, skip it to avoid duplicates ¹¹ .
12. Accumulate new events in a list (in memory). Once done reading the file (or after a certain batch size), **bulk insert** them into DuckDB. We have two strategy options: a) Use DuckDB's Python API to call `READ_JSON` or `COPY` directly on the file – DuckDB can read a JSONL file natively and

ingest it into a table in one command ¹³. This is extremely efficient in C++ under the hood. We'd need to ensure the DuckDB JSON extension is available (it typically is, by default in recent DuckDB versions). b) Alternatively, since we already parsed JSON and have Python objects, we could use `duckdb.insert` or `executemany` with parameterized SQL. This is simpler to implement but less efficient for very large batches. For MVP scale (tens or hundreds of events at a time), it may be fine. However, given our goal of robustness, using the native JSON reader is preferred if feasible, as it's optimized for bulk loading.

13. Insert into `events` table. Also, for each inserted event, insert its `event_id` into `processed` table (the `processed` table can have schema `(event_id TEXT PRIMARY KEY, ts_ingested TIMESTAMP)`). If using the unique constraint approach, `event_id` could just be a primary key in `events` itself, but since `events` might not enforce uniqueness (if we want to allow same event id from different sources? Typically `event_id` should be globally unique, so we can also make it PK in `events`).
14. Commit the transaction. We should wrap ingestion in a transaction so that if anything fails mid-way, partial inserts are rolled back (DuckDB supports transactions).
15. Print or log "ingest ok" with counts for confirmation. After implementing, test by running on the seed file provided (which has 2 events) and verify the database state matches expectations. *Output:* `ingest/ingestor.py` (or similar) that can be invoked to process new events. Tables `events` and `processed` created/updated in the DuckDB file. Verified that running it twice on the same data does not duplicate events.

16. Develop Karma Scoring (Step 5.3): Create the logic to calculate Karma v0:

17. Determine which events contribute to Karma. Based on the problem description, likely events like "eid.minted" (an identity created) or "mission.done" (a task completed by an identity) should affect that identity's score. Perhaps other event types (like contributions, approvals, etc.) would also be counted in the future. For MVP, we'll use the two types from sample.
18. For each relevant event, compute a score contribution. The prompt suggests: $\text{base } 1.0 \text{ per event} + 0.6 * \text{number of witnesses (i.e., each witness adds 0.6)} + 0.3 * \text{frequency}$ (the meaning of frequency here might be a flat 0.3 per event occurrence, or some bonus if multiple events in a short time – it sounds like simply a constant bonus per event). So for example, event "eid.minted" with 2 witnesses gives $1.0 + 0.6*2 + 0.3 = 2.5$ points (before decay).
19. Apply **EWMA decay**: We use an exponential decay factor to reduce contributions from older events. In practice, we might choose a half-life (λ) such that $\text{Score} = \text{raw_score} * \exp(-\lambda * \text{age_in_days})$. The code snippet uses `math.exp(-LAMBDA * age_days)`. For MVP, we can set λ in config (say 0.1 as in example code, meaning a half-life of about 6.93 days since $\exp(-0.1*6.93) \approx 0.5$). This means an event ~1 week old contributes half as much to Karma as an event today. Recent events thus have strong influence, encouraging ongoing activity.
20. Aggregate scores per identity (the identity id might be part of `event.key` or in payload; in sample, `payload.id` carries `eid:alice:1`). Sum up contributions of all events for each identity.
21. Store the results in the `karma` table. We can either recompute from scratch each time (for simplicity, given event log isn't huge) or do incremental updates (more complex). MVP can do a full recompute each run: delete existing karma, then insert new totals. DuckDB can compute this via SQL as well (a GROUP BY query on events, if we write a custom function for decay). But implementing in Python might be straightforward too. We will also store a textual `rationale` like "v0:witness+freq+ewma" to indicate the formula used, in case we introduce v1 later.
22. Ensure at least one example identity gets a score > 0 as expected (test with sample events).

23. Document the formula and parameters in the config and readme, including an explanation of **why** we chose EWMA (to favor recent contributions) and how **witness weighting** works (trusted witnesses add credibility, hence weight).
24. Additionally, note in documentation the plan to prevent gaming: e.g., if someone spams events, frequency adds points but witnesses are required to validate, and a human review of high scorers is implied. (This ties to Goodhart's law mitigation by adding qualitative input ¹⁹.) *Output:* `karma/calculator.py` that can be run after ingestion to update the Karma table. Confirmed that running it yields expected values for test data (we can log the top N scores as a sanity check).
25. **Generate Reports & Telemetry (Step 5.4):** Implement daily report generation:
26. Create a function to produce the Markdown content. This function will likely query DuckDB for:
 - Total events ingested *today* (or in the last 24h) – or possibly total events overall, but “daily report” implies focusing on that day. We have a date field or can derive date from timestamp.
 - Number of certain event types (e.g., how many `mission.done` today, how many `eid.minted`, etc. – these could be listed as interesting metrics).
 - Perhaps list the top 3 identities by Karma (overall, or new in that day).
 - Any notable changes (if we store previous day's Karma, we could list identities whose score increased the most today – but that might be advanced; MVP could skip this).
 - Possibly an “operational proposal” section – this could be a placeholder where the Ops Copilot suggests something, for instance: if an identity's Karma dropped due to inactivity, propose re-engaging them, or if an SLO is violated, flag it. Since AI logic is beyond MVP, we might simply put a static message or a simple rule-based note for now.
27. Format these into a Markdown template. Use bullet points for clarity as per formatting guidelines.
28. Filename as `reports/daily/YYYY-MM-DD.md` (ISO date).
29. Write the file. If publishing to IPFS is configured, invoke the IPFS CLI (e.g., using `subprocess` to run `ipfs add <file>` and capture the CID). Then perhaps include the CID in the report or log it.
30. Append to `memoria/witness.jsonl` a JSON line with `{ kind: "daily_report", date: "...", events: X, karma: Y, ... }`. This log grows indefinitely; it serves as an audit trail that the daily report was generated and what the high-level numbers were.
31. Also calculate any **SLI metrics** for the run: e.g., measure how long ingestion+processing took (duration), how many events were invalid (validation error count), etc. These could be printed to console or logged. Over time, these could be fed into a monitoring dashboard. For MVP, we ensure the code collects these stats so they can be manually reviewed.
32. This step essentially closes the loop for one day's run. We test it by running after the ingestion and karma steps, and check that the Markdown is correctly populated and the witness log updated. *Output:* A new file in `reports/daily/`, the updated witness.jsonl log. Also console output “daily ok” as in the example code, indicating completion. In documentation, instructions on how and when to run this (maybe the pipeline script runs ingestion then karma then report all in one go daily).
33. **(Optional) IPFS Publication (Step 5.5):** Although optional, we prepare a script or instructions for publishing reports to IPFS:
34. Ensure IPFS daemon (Kubo) is installed and running, or use an IPFS HTTP API.

35. The script would take a file (the daily report or even a tarball of the whole reports directory) and call `ipfs add`. This returns a CID (Content Identifier).
36. Optionally, pin the CID (to persist it on the node or a remote pinning service).
37. Log or output the CID so it can be shared or looked up on an IPFS gateway.
38. Document how to retrieve the report via the CID (for example, using a public gateway URL like `https://ipfs.io/ipfs/<CID>`).
39. Since this is optional and not a core acceptance criterion, we won't fail the MVP if IPFS is not used; it's more of a bonus feature to demonstrate forward-thinking design. *Output:* A small utility (maybe `publish_ipfs.py`) and documentation in the README on how to use it. Possibly a config flag to enable automatic IPFS publishing after report generation.
40. **Plan On-chain Integration (Step 5.6 for next iteration):** While we won't implement full blockchain connectivity in MVP-01, we will **design the interface** for it and possibly include a stub:
 41. Identify how to get events from a Tendermint (CometBFT) blockchain. Likely via subscribing to a websocket or RPC for block events. We note that in Tendermint, events might include **proposal, prevote, precommit steps** for consensus ²⁰, as well as transaction events. For our use case, relevant on-chain events might be when an identity NFT is minted or when a mission NFT/token is transferred, etc.
 42. We outline a module (maybe `ingest/onchain.py`) with a class or function that could connect to a node and listen for certain event types. We document what events would be relevant (e.g., an event `eid.minted` might actually originate from a smart contract emitting an event when a new identity token is created).
 43. Provide a configuration placeholder for node RPC address, and perhaps a way to simulate/test (since actual chain integration might require a running blockchain environment).
 44. Emphasize in documentation that **on-chain events are not required for this MVP delivery** but the system is designed to accommodate them. The **schema** already has `source: "chain"` as an option, so any event from the blockchain would be marked as such. The idempotency via `event_id` would also apply (on-chain events likely have a transaction hash or a composite ID of block-height + tx-index + event-index to uniquely identify).
 45. By planning this, we ensure the developer who continues with iteration 2 knows where to hook in this functionality without redesigning the whole system. *Output:* Design notes or pseudo-code in the repository for blockchain integration, ensuring it's clear how to extend the MVP.

After these steps, we will have a working MVP. We will then conduct the QA validation as per the acceptance criteria to make sure everything is in order.

Market Potential of the MVP

The **MRV & Ops Copilot** addresses needs at the intersection of **Web3 (blockchain) communities and DevOps/Analytics**. There is significant potential for a tool like this in various markets:

- **DAO Governance and Reputation Systems:** In decentralized autonomous organizations (DAOs), tracking member contributions and reputation is crucial. Our Karma scoring system is essentially a **reputation score** that aggregates on-chain and off-chain contributions with witness validation. This concept is already emerging – for example, the Karma project by KarmaHQ provides reputation scores for DAO contributors using weighted metrics from on-chain voting, forum activity, etc. ²¹. This validates a strong market need: DAOs want to **identify and reward active, credible contributors**. Our MVP could evolve into a similar reputation dashboard

tailored to our specific community (Esentya/Essenthius) or offered as a product to other communities. The advantage of our approach is its extensibility (easy to add new event types) and transparency (open JSON Schema, audit logs).

- **DevOps and Software Teams:** The inclusion of DORA metrics and SRE practices means Ops Copilot can also serve **software engineering teams** looking to monitor their pipeline health. Many companies invest in tools to collect deployment stats and incident metrics (Google's Four Keys is an open example ⁷ ⁸). Our MVP could be pitched as an **internal tool for engineering productivity**: it ingests events from CI/CD (builds, deploys) alongside other operational events, and produces daily health reports. With further development, it could integrate directly with GitHub or CI systems to log deployments, then calculate DORA metrics automatically. This would interest any tech organization aiming to improve DevOps performance.
- **Project Management & AI Ops Assistants:** "Ops Copilot" suggests a broader vision of an AI assistant for operations. The MVP's data collection and reporting foundation could be the first step towards a smart assistant that not only reports metrics but also analyzes them and makes suggestions (e.g., "Your deployment frequency dropped this week, perhaps due to X; consider optimizing Y"). The market for **AI Ops (AI for IT operations)** is growing – big players like Microsoft's Copilot in various domains or startup tools in monitoring are evidence of this trend ²² . Our system, with continued development (especially adding ML/AI for anomaly detection or suggestions), could enter the AI Ops market. Even in its MVP form, it provides value by automating data aggregation and informing decisions daily, which is a step above static dashboards.
- **Blockchain Infrastructure Monitoring:** Specifically for blockchain networks (if Esentya/Essenthius relates to a blockchain project), there's a need for tools that combine **on-chain metrics (like transactions, consensus events)** with off-chain context. For example, a blockchain operator might want to know not just technical metrics but community engagement (issues resolved, proposals made, etc.). MRV & Ops Copilot could fill a niche by bridging those – e.g., a validator could see a daily report of chain health (blocks minted, any consensus rounds issues) plus ops metrics (node updates deployed, etc.). Few tools currently offer a unified view; most either focus on chain analytics or on devops, but not both.
- **Consulting and Integration Potential:** Because our solution is built on open standards and is self-contained, it can be relatively easily integrated into other systems or customized. This means aside from a product, there's potential in **services/consulting** to adapt the Copilot for different use-cases. For instance, a company might want a similar pipeline but for their own custom events – since our core is generic (just needs schema and rules adjustments), we could cater to that. The market potential extends to any scenario where **traceability and daily insight into operations** are valued (e.g., supply chain tracking, environmental monitoring where MRV stands for Monitor, Report, Verify as in carbon accounting, etc.). Our architecture could be repurposed there with minimal changes, since JSONL + DuckDB is general-purpose.

In summary, the MVP positions us at a **convergence of DevOps analytics and decentralized governance tools**, which is relatively unique. By demonstrating the MVP in our context, we can attract interest from communities looking for an **all-in-one insights generator** that leverages both on-chain trust signals and off-chain data. The fact that it's daily and automated means it can become a routine part of decision-making (much like daily stand-up reports or dashboards are). With further development, this could evolve into a full "Copilot" that not only informs but also interacts (perhaps via chat interface or alerts). The market trend toward **automation and AI assistance** in operations suggests that a successful MVP could be the seed of a larger platform, giving us a competitive edge if we develop it into a product.

Sustainability and Scalability

From both a technical and a project perspective, we have designed the MVP with **long-term sustainability and scalability** in mind:

1. Technical Sustainability (Maintainability): The use of standard formats and well-supported libraries ensures that the project will be easy to maintain and extend: - *Open Standards & Open Source:* We rely on standardized formats like JSON and JSON Schema, and open-source tools (DuckDB, Pydantic, etc.). This means we are not locked into proprietary systems and can update components as needed. JSON Schema Draft-07, for instance, will remain interpretable for years and can be upgraded to newer drafts gradually if needed (most of our validation logic can adapt because libraries like Ajv note draft-07 as the most widely used stable version ³). DuckDB is MIT-licensed and built for longevity with a growing community and even a non-profit foundation to ensure its viability ²³ ²⁴. We can trust that these components won't suddenly disappear or become unsupported. - *Modular Architecture:* The project is divided into clear modules (ingest, karma, reports, etc.), which encourages sustainable development. If one part needs to change (say we adopt a different database or add an AI module), we can do so without rewriting everything. This separation of concerns also aids in testing each part independently. - *Configurable and Extensible:* Key parameters (weights for scoring, list of event types) are in configuration files. To adjust the system (e.g., change the decay rate or include a new event type in Karma calculation), one can update config or extend the schema rather than modifying core code. This reduces the risk of introducing bugs for simple changes and allows non-developers (with some technical knowledge) to fine-tune the system. - *Logging and Transparency:* The audit trail (`witness.jsonl`) and verbose logging of operations ensure that if something goes wrong, maintainers can trace back through what the system did. Transparency of data and calculations (we store the rationale for Karma and keep all events) means debugging and verification is straightforward – a new developer can recompute things or check the logs to understand system behavior. This is a sustainable practice as the project grows, preventing “black box” syndrome. - *Documentation (Runbook):* By providing a detailed runbook with SLOs, troubleshooting, and design rationale, we make sure future team members (or the dev taking over) understand the “why” and “how”. This knowledge sharing is critical to sustainable development, so the project doesn't become dependent on a single person's memory. - *Quality and Testing:* Though not explicitly mentioned earlier, as part of sustainability we should include some basic tests (even if manual) for the components. For example, unit tests for the Karma calculation (with known inputs and expected outputs) and integration test for ingestion (ingest a fake file and query the DB). Investing in tests early means safer refactoring later, contributing to long-term code health.

2. Scalability (Technical Growth): We anticipate the system will need to handle **increasing load and new requirements**. Here's how our design scales: - *Data Volume:* DuckDB's capacity to comfortably handle **up to billions of rows on a single machine** means our event log can grow quite large before encountering issues ¹². If we ingest, say, 10,000 events per day (~3.65 million a year), DuckDB can still manage that on a typical laptop given its vectorized engine and use of disk for larger-than-memory data ¹² ²⁵. If needed, we can prune or archive old events (since we might not need second-by-second detail from years ago, or we can aggregate them). Also, DuckDB's single-file makes backups or cloud sync straightforward. - *Performance:* Using bulk operations and set-based processing (SQL) is inherently scalable. If the event rate increases, our approach of COPY/bulk insert will scale much better than naive row-by-row inserts. The Karma calculation via aggregation will also scale linearly. Should performance start dipping, DuckDB allows easy optimization (adding indexes if needed, though often scans are fine) and even the option to connect to external storage (S3 etc.) if data outgrows local disk. We could also consider **MotherDuck** or **DuckDB Cloud** offerings in the future for seamless scaling, without code changes. - *Event Sources:* The design accommodates new event sources simply by adding new values to `source` (the schema enum) and handling their events in ingestion. For example, if we integrate a GitHub webhook for pull request events, we just need to convert those into our JSONL format and drop

into `bus/`. The rest of the pipeline (validation, storage) works as-is. This **scales functionally** – the system can grow in scope to cover more types of operational data (CI builds, monitoring alerts, etc.) without significant refactoring. - *User Load*: Currently, it's a single-user or team internal tool (generating reports for the team). If we wanted to scale to many users (e.g., multiple teams each getting their own reports), we could containerize the application and deploy multiple instances (since it's file-based, each instance could operate in its own namespace/directory). The modular design also means we could separate components if needed – for instance, run ingestion continuously in a service, and run report generation daily via a scheduler like cron or Airflow. This separation could handle higher frequency events or more complex computations by distributing workload. - *Scaling the Dashboard*: Metabase connecting to a local file is fine for one team; if the user base grows or remote access is needed, we can host the DuckDB file on a server or use DuckDB's new server mode (or even import the data into a larger warehouse like BigQuery if ever needed, since it's just SQL data). Because our data model is simple and based on SQL, migrating or scaling up to a bigger database is straightforward (we could even put DuckDB behind an API for real-time queries). In essence, we are not locked in; we can scale up vertically (bigger machine) or even transition to a distributed setup if absolutely required (though as noted, likely unnecessary for < billions of rows scenarios ¹²). - *Algorithmic Scalability*: The Karma algorithm and daily report are currently simple (linear scans and group-bys). As we potentially incorporate more complex analysis (maybe social network analysis on witnesses, or AI/ML recommendations), we have to ensure those scale. We can leverage DuckDB for heavy computations (SQL can do surprisingly complex things efficiently, and it can be extended with custom functions if needed). If we integrate ML, we might use embeddings or models – DuckDB has extensions for that too, or we can offload to Python. In any case, our current pipeline leaves room for inserting such steps (e.g., after computing Karma, one could run a Python ML model to classify something). We've kept data accessible and in a structured form, which is key for scaling algorithmic complexity.

3. Project Sustainability (Future Development & Community): Beyond just code, a sustainable project should consider how it remains useful and funded: - *Clear Roadmap*: We have outlined next steps like on-chain integration and improving the Karma model. By having a roadmap, we can attract contributions or justify further investment, because stakeholders see a plan. For example, one week after MVP we plan to integrate blockchain events; going forward, we might plan a v2 of Karma using more advanced metrics or even community input weighting. - *Community & Open Collaboration*: If appropriate, open-sourcing this project (once stable) could invite community contributions, especially since many DAOs or dev teams might want a similar tool. Being based on permissive licenses (DuckDB MIT, likely our code can be MIT or Apache) is inviting. A community around the tool can share best practices (like how to calibrate λ for decay using real data, which we intend to do as soon as we have more events – **calibrating EWMA half-life with real usage patterns will be important**). - *Sustainable Business Model*: If this MVP is for an internal project, sustainability might mean demonstrating value so it continues to be used and improved. If it's intended as a product, we should consider how it could generate value or revenue. Possibilities: as a **service offering** (e.g., hosted Ops Copilot for clients), or as part of a bigger platform (maybe Essentius has a platform where this is a feature). Because the cost to run it is low (no expensive infra needed, runs on a single machine or cheap VM), it's economically sustainable. If scaling out to many users, we'd evaluate a multi-tenant setup or cloud architecture, but until then the overhead remains minimal. - *Scalability of Team*: As the project grows, more developers might get involved. Our emphasis on documentation, standards, and modularity ensures new team members can onboard quickly. We treat the runbook and code comments as living documents. Also, by using common languages (Python/SQL) and avoiding obscure tech, it's easier to hire or assign devs to work on this. Python is widely known, and the patterns we use (JSON schema, ETL, SQL analytics) are common skill sets. - *Environmental Sustainability*: A minor but noteworthy point – using single-node processing (DuckDB) is actually **more energy-efficient** than running large clusters for moderate data ²⁶ ²⁷. DuckDB's philosophy (do more on one node that's usually idle) helps reduce the carbon footprint for our analytics. If our project were to scale massively, we'd continue to weigh the cost of

distributed systems; maybe we can handle most needs on modern hardware without needing an energy-hungry cluster. This aligns with a trend in big data towards leveraging powerful single machines for 99% of use cases ¹² ²⁵ . So, in a way, our approach is sustainably using resources.

In conclusion, this MVP is built not just to solve an immediate problem, but to do so in a way that **can grow and adapt**. The choices of technology and design patterns were made to keep the system **robust, flexible, and scalable** for future demands. Whether it's handling ten times the data, integrating new event types, or being repurposed for a new market, we are confident the foundation laid by MRV & Ops Copilot will support it. We've mitigated key risks (duplication, performance bottlenecks, metric gaming) through design, and we have a clear path to evolve the project (on-chain data, smarter scoring, etc.). This ensures that the MVP will not be a throwaway prototype, but rather the **first step of a lasting, scalable solution** in the space of operational intelligence and reputation tracking.

Sources:

- NDJSON (JSON Lines) specification – ensures one JSON event per line ²
- Martin Fowler on Event Sourcing – inspiration for our append-only event log design ¹⁴
- Idempotent Consumer pattern – to avoid duplicate processing of events ¹¹
- JSON Schema Draft-07 widely supported and used in tooling ³
- Pydantic for JSON schema generation and data modeling ⁴
- DuckDB JSON bulk load (COPY FROM) – efficient ingestion of newline JSON ¹³
- DORA “Four Keys” DevOps metrics (Google Cloud blog) ¹⁷ ⁸
- Goodhart’s Law definition – caution against blindly optimizing the Karma metric ¹
- SRE concepts of SLI/SLO and error budget (reliability trade-offs) ⁶
- Karma (karmahq) for DAOs – shows market demand for weighted contribution scores ²¹
- DuckDB capabilities – single-node handling of large datasets (~1 billion rows) ¹²
- DuckDB efficiency vs. big data clusters – leveraging modern hardware for analytics ²⁵

¹ ¹⁹ Goodhart's law - Wikipedia

https://en.wikipedia.org/wiki/Goodhart%27s_law

² ⁹ ¹⁸ NDJSON Format Specification: Complete Technical Guide - JSONL Tools

<https://jsonltools.com/ndjson-format-specification>

³ Choosing schema language | Ajv JSON schema validator

<https://ajv.js.org/guide/schema-language.html>

⁴ JSON Schema - Pydantic Validation

https://docs.pydantic.dev/latest/concepts/json_schema/

⁵ 6.3.2.4. EWMA Control Charts - Information Technology Laboratory

<https://www.itl.nist.gov/div898/handbook/pmc/section3/pmc324.htm>

⁶ SRE: SLIs, SLOs, Error Budgets & Their Application | by Shreyasingh | Medium

<https://medium.com/@shreyasingh0105/sre-slis-slos-error-budgets-their-application-05d5ace3f22d>

⁷ ⁸ ¹⁷ Use Four Keys metrics like change failure rate to measure your DevOps performance | Google Cloud Blog

<https://cloud.google.com/blog/products/devops-sre/using-the-four-keys-to-measure-your-devops-performance>

¹⁰ ¹¹ Pattern: Idempotent Consumer

<https://microservices.io/patterns/communication-style/idempotent-consumer.html>

12 23 24 25 26 27 **DuckDB: the Rise of In-Process Analytics and Data Singularity | endjin - Azure Data Analytics Consultancy UK**

<https://endjin.com/blog/2025/04/duckdb-rise-of-in-process-analytics-understanding-data-singularity>

13 **JSON Import – DuckDB**

https://duckdb.org/docs/stable/guides/file_formats/json_import.html

14 15 **Event Sourcing**

<https://martinfowler.com/eaDev/EventSourcing.html>

16 21 **How Karma reputation scoring works?**

<https://www.karmahq.xyz/how-it-works>

20 **What is Tendermint**

<https://docs.tendermint.com/v0.34/introduction/what-is-tendermint.html>

22 **Overview of Copilot capabilities in finance and operations apps**

<https://learn.microsoft.com/en-us/dynamics365/fin-ops-core/fin-ops/copilot/copilot-for-finance-operations>